

Act 4: The AES-256 Starting Key Matrix

The shortest Act by a long way. Everything hard is already done - Bob and Alice both hold the same 32 bytes. This Act does one thing: puts those bytes into the shape AES expects.

The code, all of it

```
aes_key_matrix = [[0] * 8 for _ in range(4)]

for byte_index in range(32):
    row = byte_index % 4
    col = byte_index // 4
    aes_key_matrix[row][col] = K_final[byte_index]
```

Why 4 rows by 8 columns

AES-256 uses a 32-byte key, and it lays that out as 4 rows by 8 columns ($4 \times 8 = 32$). Each COLUMN is 4 bytes, which AES calls a word. So the key is 8 words wide.

The ordinary AES state block is 4×4 . The key here is twice that width because a 256-bit key is twice the size of one 128-bit block.

The fill order - DOWN each column, not across each row

byte 0 ↓	byte 4 ↓	byte 8 ↓	byte 12	byte 16	byte 20	byte 24	byte 28
byte 1 ↓	byte 5 ↓	byte 9 ↓	byte 13	byte 17	byte 21	byte 25	byte 29
byte 2 ↓	byte 6 ↓	byte 10 ↓	byte 14	byte 18	byte 22	byte 26	byte 30
byte 3 ↓	byte 7 ↓	byte 11 ↓	byte 15	byte 19	byte 23	byte 27	byte 31

Byte 0 goes to row 0 column 0. Byte 1 to row 1 column 0. Byte 2 to row 2, byte 3 to row 3 - filling the first column top to bottom. Then byte 4 starts the next column back at row 0, and so on across all 8 columns.

That is exactly what the two lines of arithmetic do:

```
row = byte_index % 4    -> cycles 0,1,2,3, 0,1,2,3, ...
col = byte_index // 4   -> advances every 4 bytes: 0,0,0,0, 1,1,1,1, ...
```

What Act 4 is, and what it is not

There is only ONE key

aes_key_matrix is not a new or separate thing. It contains exactly the same 32 bytes as K_final, just drawn out in a grid instead of a line. Read the grid back down each column and you get K_final again, byte for byte identical.

K_final IS the AES starting key. The matrix is a picture of it.

Where this matters - and where it does not

IF BUILDING AES FROM SCRATCH: this grid is genuinely the starting point. The AES key schedule works column by column on exactly this layout, expanding these 8 columns into the 60 columns needed for all 15 round keys.

IF USING A LIBRARY: the library wants the plain 32 bytes and builds its own internal layout. It never sees this grid. In that case Act 4 demonstrates that you understand the format, but it is not a step the library actually performs.

Worth being straight about that distinction in a write-up rather than implying the matrix is doing work it is not.

The handover point

Act 4 is where ML-KEM finishes and AES begins. Everything before it - the lattices, the noise, the NTT, the compression, the re-encryption check - existed for one purpose: getting two strangers to hold the same 32 bytes without ever sending them.

From here on the maths is completely different and belongs to AES, not Kyber.

What is NOT built yet

- the AES key schedule (turning these 8 columns into 15 round keys)
- the AES cipher itself (SubBytes, ShiftRows, MixColumns, AddRoundKey)
- encrypting or decrypting an actual message

Act 4 produces a correctly shaped key and stops there. Nothing has been encrypted.